ENSF 338

TERM PROJECT REPORT

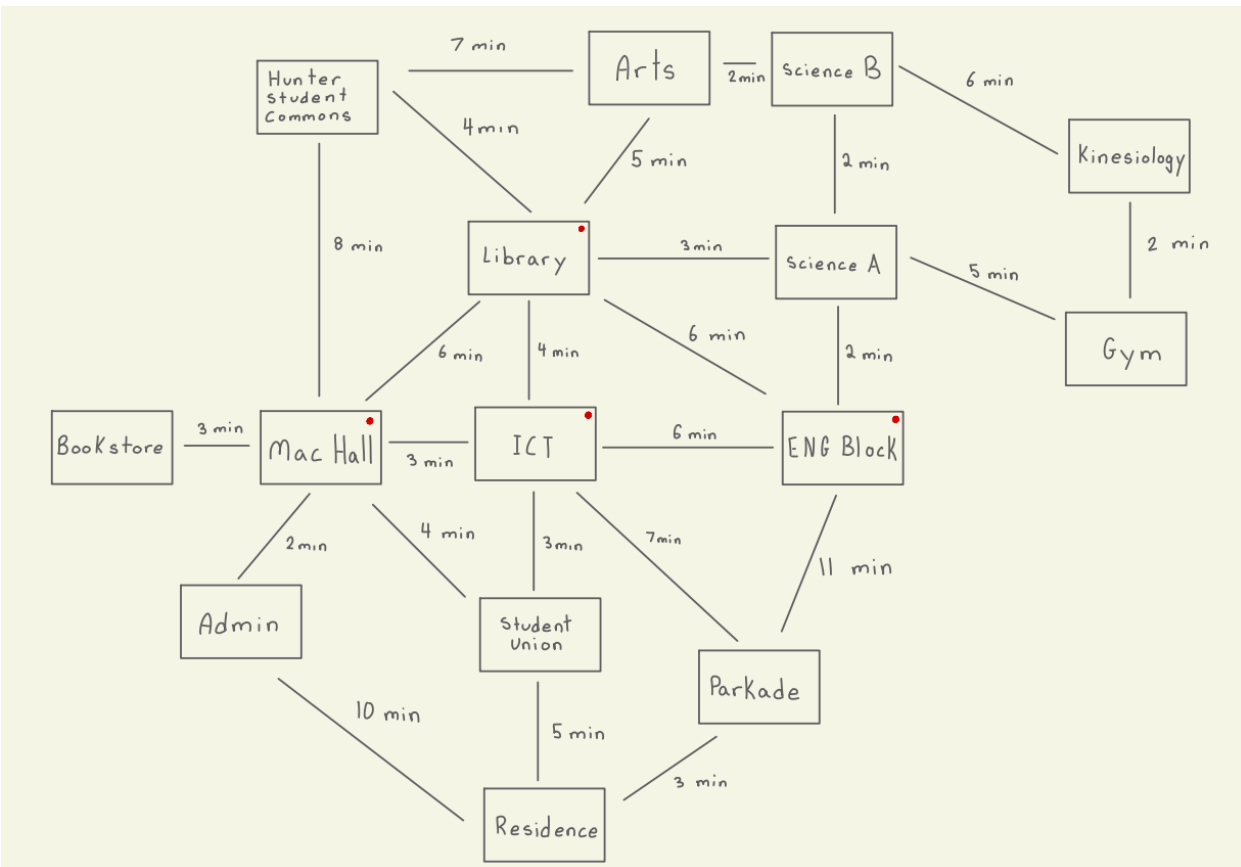
Group 09 Members:
Reyansh Badhwar (30244010)
Abdelrahman Attia (30240604)
Himaal Ishaq (30249404)
Abiya Raheel (30242511)
Alina Kobrusev (30248512)
Nayha Rehman (30250126)

Link to GitHub Repo:
https://github.com/reyanshBadhwar26/ensf338-finalproject.git
Submission Date: April 15th 2026

Introduction:

Our project presents a Campus Navigation and Event Management System designed to model our university campus in a practical and organized way. By implementing multiple features using the various types of data structures we learned in this course, we combined navigation, request processing, and event management into a working application that is both simple to understand and efficient to work with. The application presents a model of campus buildings and pathways, uses Dijkstra's algorithm to compute shortest routes, manages room bookings, provides efficient lookup of campus resources, and allows priority-based service handling. To keep the system clear and maintainable, we adopted a modular design, keeping all of our code modules connected through the central Campus model.

Our campus map included a total of 15 buildings (campus nodes) and 25 edges (pathways with time weights) and was modelled as follows:



Design Decisions & Complexity Analysis:

Feature 2.1 - Campus Map and Shortest Path Navigation:

Design Decisions:

To implement campus navigation, we first developed a map-loading component which reads the campus file and constructs the campus as a weighted undirected graph. In this representation, each building is represented as a node, and each pathway between two buildings is represented as an edge weighted by travel time in minutes.

Given the source and destination building, the system then computes and displays the shortest path using Dijkstra's algorithm. This algorithm was chosen because the campus map is a weighted graph with non-negative edge weights, making it appropriate for finding the minimum total travel time between two locations. To support the program, we implemented a min-heap that always selects the next building with the smallest currently known distance. This allows Dijkstra's algorithm to efficiently compute the shortest route, after which the system displays both the sequence of buildings in the path and the total travel time. To meet the project requirements, we tested on a map with 15 campus buildings and 25 weighted pathways.

Complexity Analysis:

Let V be the number of campus buildings (nodes), and E be the number of pathways (edges).

Map Loading:

Best: O(E)
Average: O(E)
Worst: O(E)

The Map-loading component reads the campus file line by line and processes each edge on entry once. For each valid line, it extracts the two building IDs and the edgeweight, creates missing buildings if needed and inserts pathways in both directions. Since each edge is handled once the overall complexity is linear in the number of edges.

Shortest path using Dijkstra's algorithm:

Best: O((E + V)log(V))
Average: O((E + V)log(V))
Worst: O((E + V)log(V))

The shortest path feature uses Dijkstra's algorithm with a min-heap. Each building and pathway may cause heap operations such as insertion and removal, and each heap operation takes

O(log(V)) time . The algorithm also checks adjacent buildings through the adjacency structure, so the total runtime is then built from vertices and edges together with heap updates.

Space complexity:

Graph Storage: O((E + V)) because the adjacency dictionary stores all buildings and all pathway connections.

Dijkstra: O (V) because the algorithm keeps supporting structures like distance, previous-node, and visited records for each building.

Feature 2.2 - Route History and Navigation Undo:

<u>Design Decisions:</u>

We implemented route history using a stack, represented with a Python list. Each navigation query is stored as one history entry containing the starting building, destination building, full path, and total travel time. The most recent route is always placed at the top of the stack (end of the list).

This approach is appropriate because:

- Users undo the most recent navigation query first, which follows a Last-In, First-Out (LIFO) pattern.
- Adding a new route to history happens frequently and can be done efficiently with a push operation.
- Undo only needs to remove the most recent route and reveal the previous one, which matches stack behaviour naturally.
- No searching or sorting of history entries is required.

An alternative considered was using a queue, but a queue follows First-In, First-Out (FIFO) order, which is not suitable for undo functionality. A list with index removal was also considered, but this would add unnecessary complexity since undo only ever works on the most recent route. Therefore, a stack was the most appropriate choice.

<u>Complexity Analysis:</u>

Let n be the number of saved navigation queries in the current session. Then:

- Add route to history:

    - Best O(1)
    - Average O(1)
    - Worst O(1)

  Adding a route uses append() on the Python list, which places the new entry at the end of the list, corresponding to the top of the stack.
- Undo last route:
    - Best O(1)
    - Average O(1)
    - Worst O(1)

  Undo performs one pop() to remove the most recent route and one peek() to access the previous route. Both operations only involve the end of the list.

- View most recent route:
    - Best O(1)
    - Average O(1)
    - Worst O(1)

  Peeking at the top of the stack directly accesses the last element of the list without traversal.

- Space Complexity: O(n), since each navigation query in the session is stored once in the stack.

Feature 2.3 - Room and Event Booking System:

<u>Design Decisions:</u>
We implemented a booking system using a combination of Python dictionaries and sorted lists to manage room reservations by date and time. The dictionaries acting as hashmaps are used to store bookings indexed by unique keys such as room, date, and start time to allow direct access to specific bookings.

This approach is appropriate because:
- It balances fast access with ordered retrieval
- Hashmaps provide efficient, constant-time average-case lookup
- Sorted lists allow for the maintenance of the chronological order of events

An alternative we considered was using a single list to store all bookings, but this would require linear search for lookups and insertions, making it inefficient as the number of bookings grows. Therefore, a combination of dictionaries and sorted lists was chosen to achieve both fast access and ordered traversal.

Complexity Analysis:

Let n be the total number of bookings currently stored in the system, d be the number of total bookings on a specific day, r be the number of bookings in a specific room on a day, and k be the number of results returned.

Add booking:
Time Complexities: Best: O(log r), Average: O(r), Worst: O(n)
Space Complexity: O(1)
Binary search finds the insertion index in O(log r), but inserting into a list requires shifting elements, leading to O(r) time.

Remove booking:
Time Complexities: Best: O(1), Average: O(r + d), Worst: O(n)
Space Complexity: O(1)
The booking is found in O(1) using a hash map, but removing it from the list requires scanning and shifting elements, which causes linear time.

Booking Retrieval:
Time Complexities: Best: O(1), Average: O(1), Worst: O(1)
Space Complexity: O(1)
The booking is retrieved using a HashMap keyed by room, date, and start time.

Range query:
Time Complexities: Best: O(k), Average: O(d), Worst: O(n)
Space Complexity: O(k)
The method scans all bookings for a given day and filters those within the time range, resulting in linear time.

Events on a given day:
Time Complexities: Best: O(d), Average: O(d), Worst: O(d)
Space Complexity: O(d)
Returning the bookings requires copying the list, taking linear time in the number of events that day.

Next upcoming event:
Time Complexities: Best: O(log n), Average: O(log n + d), Worst: O(n)
Space Complexity: O(1)
Binary search is used to locate the relevant data and time, but multiple days may be scanned in the worst case.

Feature 2.4 - Priority-Based Service Queue:

Design Decisions:

We implemented a PriorityServiceQueue to manage campus service requests handled based on three levels of urgency rather than arrival time alone. Therefore, according to the instructions, our design uses three separate queues: emergency_queue, standard_queue, and low_queue. We chose to implement each priority class as a linked-list Queue class, as we only needed to support insertion at the rear and removal at the front. This design also naturally supports FIFO order within each priority level to keep our design efficient and manageable.
Each incoming request is represented by a ServiceRequest object. This stores the request ID, service type, description, priority level and requester name. The service_queue object is stored in the Campus class to integrate this service into the larger program.

Alternatives Considered:
One of our initial considerations was using binary heaps to store the service requests. This would support priority ordering, but it would not preserve arrival order among requests with the same priority levels without extra modifications. This would add more complexity and unnecessary fields. We also considered using single sorted lists for this task, but quickly rejected it as inserting new requests into their required positions would require shifting elements. This would increase the complexity and make the whole process of insertion more inefficient. Therefore, we settled on using the Queue design shown currently.

Complexity Analysis:

Let n be the total number of service requests in the system.

add_request
Time Complexities: : best: O(1), average: O(1), worst: O(1).
Space Complexity: O(1)
This method only checks the priority of the request and enqueues it to the appropriate linked list queue. Because this only enqueues a node to the rear of the queue, no traversal is required.

serve_next_request:
Time Complexities: O(1), average: O(1), worst: O(1)
Space Complexity: O(1)
This method checks the three queues and dequeues from the first non-empty one. Because this only removes the front node directly, no traversal is required.

peek_next_request:
Time Complexities:O(1)
Space Complexity: O(1)
This method simply checks the front of the queues and returns the first available request.

is_empty:
Time Complexities:O(1)
Space Complexity: O(1)
This method only evaluates if each queue is empty or not.

total_requests:
Time Complexities: O(1)
Space Complexity: O(1)
Each queue already stores its current size, so this method simply adds these three values to count the total number of requests.

display_queue:
Time Complexities: O(n)
Space Complexity: O(n)
This method traverses all requests currently stored in the queues, so it can print them. In order to do this, each queue is converted to a list, and so this has a total time complexity of O(n)

Feature 2.5 - Fast Building and Resource Lookup:

<u>Design Decisions:</u>

We implemented the lookup system using Python dictionaries, which act as hash tables. Buildings and rooms are stored using unique identifiers (building_id, room_id) as keys, allowing direct access to their data.

This approach is appropriate because:

- It supports fast lookups, which are frequently required by users.
- Insert and delete operations are efficient and simple.
- No ordering is required, making hash tables more suitable than ordered structures.
- Performance remains nearly constant as the dataset grows.

An alternative considered was using a list, but this would require linear search O(n), making it inefficient for large datasets. A binary search tree was also considered, but it has O(logn) performance and added complexity compared to hash tables.

Complexity Analysis:

Let n be the number of stored records.

- **Insert:** Best O(1), Average O(1), Worst O(n)
- **Lookup:** Best O(1), Average O(1), Worst O(n)
- **Delete:** Best O(1), Average O(1), Worst O(n)

Python dictionaries use hashing to compute an index for each key, allowing direct access in constant time in the best and average cases. In the worst case, many hash collisions can occur, causing operations to degrade to linear time.

- **Space Complexity:** O(n), as each record is stored once.

Feature 2.6 - Incoming Request Processing:

Design Decisions:

We implemented the incoming request processing using a 2.4's linked list queue implementation. This gave us a Queue with enqueue(request) for enqueue and dequeue() for dequeue. A queue is used so that we can handle requests in arrival order (FIFO), so the oldest request will always be processed first. A linked list was chosen over a list because removal from the front is O(1) (compared to a dequeue of O(n) for a normal Python list) since we do not have to shift any elements, making it more efficient for queue operations.

Complexity Analysis:

Let n be the number of incoming requests.

- **Enqueue:** Best O(1), Average O(1), Worst O(1)
- **Dequeue:** Best O(1), Average O(1), Worst O(1)
- **is_empty:** Best O(1), Average O(1), Worst O(1)

Since enqueue adds to the end of the linked list, updating the tail pointer is O(1). The same is true for dequeue, where we remove an element from the front of the list (i.e. removing the head), so we only have to update the head pointer (again is O(1). Is_empty only checks if the head is empty; this operation is always O(1).

- **Space Complexity:** O(n), as each request is stored once.

Feature 2.7 (Bonus) - Balanced Event Index:

Design Decisions:

We implemented the balanced event index using a separate AVL tree class, booking_avl.py, and integrated it into the existing room booking system as an additional index rather than replacing the structures already used for Feature 2.3. Each AVL tree node stores a booking and uses a composite key of (event_date, start_time, room) so that bookings are ordered chronologically, with room as a tiebreaker.

This approach is appropriate because:

- It guarantees that insert and lookup operations remain efficient even when bookings are inserted in adversarial order, such as chronological order.
- The AVL tree automatically maintains balance through rotations, preventing the index from becoming a skewed linear chain.

We added a print_tree() method that provides a direct way to visualize the tree structure and verify that the balance property is maintained after insertions.

An alternative considered was using a regular binary search tree, but a normal BST can become unbalanced when bookings are inserted in sorted chronological order, causing insert and lookup to degrade to O(n).

Complexity Analysis:

Insert: Best, Average and Worst - O(logn)
Insertion first follows the binary-search-tree path from the root to a leaf, then performs at most a constant number of AVL rotations to restore balance. Because the tree remains height-balanced, its height is always O(logn).

Lookup/Search: Best - O(1), Average and Worst - O(logn)
In the best case, the booking is found at the root, while in the average and worst cases, the search follows the height of the AVL tree, which is O(logn)

Delete: Best, Average and Worst - O(logn)
Deletion locates the target booking, removes it using standard BST deletion rules, and then rebalances the tree if necessary.

Space Complexity: O(n), since each booking stored in the balanced index occupies one AVL tree node.

Challenges & Lessons Learned:

- A challenge faced was handling inconsistent user input (e.g., lowercase, spaces, missing underscores, etc.), which caused lookup failures even when the data existed. The lesson learned was to implement input normalization and validation, as it's essential for reliable systems, especially when using hash tables where exact key matching is required.

- Another challenge was structuring the menu so that different features (e.g., lookup, insert, delete, navigation) interacted correctly without creating redundant logic. The lesson learned was that breaking functionality into modular helper functions (like separate handlers for each menu option) makes the system easier to manage and debug.

- One useful lesson that we learnt was seeing how different data structure choices affect complexities and maintainability for different features. In part 2.4, we implemented the priority-based service queue using separate linked-list-based queues for the different priority levels. In comparison, another part of the project considered a Python list for queue behaviour in part 2.6. We were able to see that using the same linked-list implementation in both places would allow for better consistency across the program and make the queue operations more aligned with the intended design.

- A challenge we faced while implementing 2.2 (route query undo) was ensuring that the functionality behaved correctly when the history became empty or contained only one entry. We added special cases for history (stack) being empty and prompted the user when this situation happened. The lesson learned here was that boundary conditions should always be considered and safeguards must be added to make sure the system doesn't crash and the user experience is maintained.

- The difficulty encountered in implementing the bonus balanced index (2.7) was adapting the AVL tree insertion algorithm learned in class into the booking software. Though the method for inserting values using case-based operations was successful as an individual process, it did not succeed at first when the operation was not performed because the balance factor was not updated properly. The lesson we learned here is that balance data is crucial for AVL tree balancing operations.

AI Use:

- Used the help of ChatGpt to research the uses and outputs of the __repr__ function for parts 2.4 (needed a good format to return data in)

what is a good method that returns String format of object?

Best choice in Python: `__str__`

Use `__str__` when you want the object to return a nice human-readable string.

Example:

Python Run

```python
def __str__(self):
    return f"{self.request_id} - {self.priority} - {self.service_type}"
```

Then:

Python Run

```python
print(request)
```

will automatically use that string.

Good options

- `__str__` → best for normal readable output
- `__repr__` → better for debugging/programmer-style output

explain format and use of __repr__ function.

`__repr__` is a special Python method that defines the **official string representation** of an object.

What it is used for

Python uses `__repr__` when:

- you print a list of objects
- you inspect an object in the shell/debugger
- you call `repr(object)`

Its job is usually to show a **clear, developer-friendly** version of the object.

General format

Python Run

```python
def __repr__(self):
    return "some string"
```

Usually that string includes the class name and important fields.

Difference from `__str__`

- `__repr__` = developer/debugging view
- `__str__` = user-friendly display view

So:

- use `__repr__` for a precise object summary
- use `__str__` for cleaner printed output for users

Easy way to remember

`repr` = **representation of the object for programmers**

If you want, I can also show you the difference between `__repr__`, `__str__`, and your current `display_info()` using your exact class.

- Used AI to assist in developing part of Feature 2.1, where it was used to help create the regular expression for edge_patterns, which parses campus map lines in the format ICT -- LIBRARY [weight=4];. Assistance was limited to finding a way to extract the two building IDs and pathway weight from the file.

I am reading a .dot file in Python and need a regular expression to match lines in this format:

ICT -- LIBRARY [weight=4];

I want to extract:

1. the first building ID
2. the second building ID
3. the weight value

Use this regular expression in Python:

python Copy

```python
edge_pattern = re.compile(r'(\w+)\s*--\s*(\w+)\s*\[weight=(\d+)\];')
```

What it extracts:

- `(\w+)` → first building ID, like `ICT`
- second `(\w+)` → second building ID, like `LIBRARY`
- `(\d+)` → weight value, like 4

- We used ChatGPT to help with debugging issues in our AVL tree implementation when it was not working properly for the insertion method. We also asked for guidance on improving the navigation history implementation when we wanted to ensure that our function handled edge cases properly (especially the undo history one). The tool was specifically instructed not to provide direct code but to explain the issues with it.

Thought for a few seconds

You are updating balances in `insert`, **but that is not enough**.

The reason is:

> the balance updates in `insert` are based on the **old tree shape before rotation**.

Once you do a rotation, the parent/child relationships change, so the balance values of the rotated nodes must also be updated **inside the rotation method itself**.

Why insert updates alone are not enough

Take a simple left-left case:

Before rotation:

Contributions:

Team Member	Contributions
Abdelrahman Attia	- Development of feature 2.1, including developing the campus map and map loader - Writing Technical Report and Demonstration Scenarios
Reyansh Badhwar	- Development of features 2.1 (shortest path navigation), 2.2 (navigation undo history) and 2.7 (self-balancing booking index) - Writing Technical Report and Demonstration Scenarios
Himaal Ishaq	- Development of feature 2.4 (priority queue implementation) - Writing Technical Report and Demonstration Scenarios
Alina Kobrusev	- Development of feature 2.6, (implemented a functionality to let requests be enqueued and dequeued in arrival order) - Writing Technical Report and Demonstration Scenarios
Abiya Raheel	- Development of feature 2.5: Implemented fast building and room lookup using hash tables (Python dictionaries), including efficient insert, delete, and retrieval operations with input normalization and validation. - Development of UI - Writing Technical Report and Demonstration Scenarios
Nayha Rehman	- Development of feature 2.3: Implemented event booking system using hash maps, adding, removing, and retrieving information on room bookings. - Writing Technical Report and Demonstration Scenarios

Demonstration Scenarios:

1. **Shortest path query: Show at least two route queries with different source/destination pairs, displaying the full path and total cost.**

```
Campus Navigation and Event Management System
---------------------------------------------
1. Shortest path navigation
2. Undo last navigation
3. Show navigation history
4. Room and event booking system
5. Priority service queue
6. Fast building / room lookup
7. Incoming request pipeline
8. Demo setup
0. Exit

>> 1


Shortest Path Navigation
----------------------------
Valid building IDs:
ADMIN, ARTS, BOOKSTORE, ENG_BLOCK, GYM, HUNTER_COMMONS, ICT, KINESIOLOGY, LIBRARY, MACHALL, PARKADE, RESIDENCE, SCI_A, SCI_B, STUDENT_UNION
Enter source building ID: arts
Enter destination building ID: ict

Shortest route from ARTS to ICT:
ARTS -> LIBRARY -> ICT
Total travel time: 9 minutes

Route saved to history.
```

```
Campus Navigation and Event Management System
---------------------------------------------
1. Shortest path navigation
2. Undo last navigation
3. Show navigation history
4. Room and event booking system
5. Priority service queue
6. Fast building / room lookup
7. Incoming request pipeline
8. Demo setup
0. Exit

>> 1


Shortest Path Navigation
----------------------------
Valid building IDs:
ADMIN, ARTS, BOOKSTORE, ENG_BLOCK, GYM, HUNTER_COMMONS, ICT, KINESIOLOGY, LIBRARY, MACHALL, PARKADE, RESIDENCE, SCI_A, SCI_B, STUDENT_UNION
Enter source building ID: gym
Enter destination building ID: student_union

Shortest route from GYM to STUDENT_UNION:
GYM -> ENG_BLOCK -> ICT -> STUDENT_UNION
Total travel time: 13 minutes

Route saved to history.
```

The above screenshots demonstrate feature 2.1, which required the implementation of an algorithm that can find the shortest route in the campus map to go from one building to another. The first route query is for going from the Arts building to ICT, which, according to the map created, had a shortest route of ARTS -> LIBRARY -> ICT with a total time (cost) of 9 minutes. The second query is for going from the Gym to the Student Union, which has the shortest route of GYM -> ENG_BLOCK -> ICT -> STUDENT_UNION, taking a total of 13 minutes.

2. **Undo navigation: Show a sequence of route queries followed by one or more undo operations, demonstrating correct state restoration.**

Continuing from the same state as above, we add a few more queries:

```
Campus Navigation and Event Management System
---------------------------------------------
1. Shortest path navigation
2. Undo last navigation
3. Show navigation history
4. Room and event booking system
5. Priority service queue
6. Fast building / room lookup
7. Incoming request pipeline
8. Demo setup
0. Exit

>> 1


Shortest Path Navigation
---------------------------
Valid building IDs:
ADMIN, ARTS, BOOKSTORE, ENG_BLOCK, GYM, HUNTER_COMMONS, ICT, KINESIOLOGY, LIBRARY, MACHALL, PARKADE, RESIDENCE, SCI_A, SCI_B, STUDENT_UNION
Enter source building ID: admin
Enter destination building ID: gym

Shortest route from ADMIN to GYM:
ADMIN -> MACHALL -> LIBRARY -> SCI_A -> GYM
Total travel time: 16 minutes

Route saved to history.
```

```
Campus Navigation and Event Management System
---------------------------------------------
1. Shortest path navigation
2. Undo last navigation
3. Show navigation history
4. Room and event booking system
5. Priority service queue
6. Fast building / room lookup
7. Incoming request pipeline
8. Demo setup
0. Exit

>> 1


Shortest Path Navigation
---------------------------
Valid building IDs:
ADMIN, ARTS, BOOKSTORE, ENG_BLOCK, GYM, HUNTER_COMMONS, ICT, KINESIOLOGY, LIBRARY, MACHALL, PARKADE, RESIDENCE, SCI_A, SCI_B, STUDENT_UNION
Enter source building ID: bookstore
Enter destination building ID: eng_block

Shortest route from BOOKSTORE to ENG_BLOCK:
BOOKSTORE -> MACHALL -> ICT -> ENG_BLOCK
Total travel time: 14 minutes

Route saved to history.
```

This shows all our queries:

```
Campus Navigation and Event Management System
---------------------------------------------
1. Shortest path navigation
2. Undo last navigation
3. Show navigation history
4. Room and event booking system
5. Priority service queue
6. Fast building / room lookup
7. Incoming request pipeline
8. Demo setup
0. Exit

>> 3


Navigation History
------------------
1. ARTS -> ICT | Time: 9 minutes | Path: ARTS -> LIBRARY -> ICT
2. GYM -> STUDENT_UNION | Time: 13 minutes | Path: GYM -> ENG_BLOCK -> ICT -> STUDENT_UNION
3. ADMIN -> GYM | Time: 16 minutes | Path: ADMIN -> MACHALL -> LIBRARY -> SCI_A -> GYM
4. BOOKSTORE -> ENG_BLOCK | Time: 14 minutes | Path: BOOKSTORE -> MACHALL -> ICT -> ENG_BLOCK
```

Applying undo operations:

```
Campus Navigation and Event Management System
---------------------------------------------
1. Shortest path navigation
2. Undo last navigation
3. Show navigation history
4. Room and event booking system
5. Priority service queue
6. Fast building / room lookup
7. Incoming request pipeline
8. Demo setup
0. Exit

>> 2


Undo Navigation
---------------
Reverted to previous navigation:
Start: ADMIN
End: GYM
Time: 16 minutes
Path: ADMIN -> MACHALL -> LIBRARY -> SCI_A -> GYM
```

```
Campus Navigation and Event Management System
---------------------------------------------
1. Shortest path navigation
2. Undo last navigation
3. Show navigation history
4. Room and event booking system
5. Priority service queue
6. Fast building / room lookup
7. Incoming request pipeline
8. Demo setup
0. Exit

>> 2


Undo Navigation
---------------
Reverted to previous navigation:
Start: GYM
End: STUDENT_UNION
Time: 13 minutes
Path: GYM -> ENG_BLOCK -> ICT -> STUDENT_UNION
```

The above screenshots show that we added a few route queries (a total of 4) in the following order:

1. ARTS -> ICT | Time: 9 minutes | Path: ARTS -> LIBRARY -> ICT
2. GYM -> STUDENT_UNION | Time: 13 minutes | Path: GYM -> ENG_BLOCK -> ICT -> STUDENT_UNION
3. ADMIN -> GYM | Time: 16 minutes | Path: ADMIN -> MACHALL -> LIBRARY -> SCI_A -> GYM
4. BOOKSTORE -> ENG_BLOCK | Time: 14 minutes | Path: BOOKSTORE -> MACHALL -> ICT -> ENG_BLOCK

This is also visible in our Navigation History portal. After this, we apply undo operations one by one. As seen in the screenshots, we first go back from our current query (Bookstore to Eng Block) to the one before it (Admin to Gym). Applying the undo operation again, we revert back one more query, reaching Gym to Student Union. This shows the implementation of required feature 2.2.

3. **Booking range query: Show a time-range query returning all bookings within a specified window.**

```
Room and Event Booking System
-----------------------------
1. Add booking
2. Remove booking
3. Get exact booking
4. Show events on a day
5. Query events in a time range
6. Show next upcoming event
7. Print booking AVL tree - Bonus 2.7
0. Back

>> 5


Query Events in Time Range
--------------------------
Date (YYYY-MM-DD): 2026-04-25
Start time (HH:MM): 00:00
End time   (HH:MM): 15:00
Booking(room='ICT-101', date=2026-04-25, 08:00:00-09:00:00, 'Hackathon')
Booking(room='ENG_BLOCK-201', date=2026-04-25, 10:00:00-11:00:00, 'Office Hours')
Booking(room='SCI_A-201', date=2026-04-25, 12:00:00-13:00:00, 'Study Session')
Booking(room='ICT-102', date=2026-04-25, 14:00:00-15:00:00, 'Study Session')
```

The above screenshot shows the implementation of feature 2.3, especially the requirement of finding all the current bookings in a specified time slot. As seen above, if we ask to see all bookings on April 25, 2026, from 12 am to 3 PM, we get 4 bookings in that time based on our test dataset.

4. **Priority queue demo: Show requests being inserted at different priority levels, then dequeued in correct priority order.**

```
Priority Service Queue
----------------------
1. Add service request
2. Serve next request
3. Peek next request
4. Display all queues
0. Back

>> 1


Add Service Request
-------------------
Request ID: REQ01
Service type: IT Support
Description: Laptop not working
Requester name: Professor Steve
Priority (Emergency / Standard / Low): Standard
Service request added.


Priority Service Queue
----------------------
1. Add service request
2. Serve next request
3. Peek next request
4. Display all queues
0. Back

>> 1


Add Service Request
-------------------
Request ID: REQ02
Service type: Cleaning
Description: Water spill in hallway
Requester name: Hallway Moniter
Priority (Emergency / Standard / Low): Low
Service request added.


Priority Service Queue
----------------------
1. Add service request
2. Serve next request
3. Peek next request
4. Display all queues
0. Back

>> 1


Add Service Request
```

```
Add Service Request
-------------------
Request ID: REQ03
Service type: Medical
Description: Student passed out in exam
Requester name: Exam moniter
Priority (Emergency / Standard / Low): Emergency
Service request added.


Priority Service Queue
----------------------
1. Add service request
2. Serve next request
3. Peek next request
4. Display all queues
0. Back

>> 1


Add Service Request
-------------------
Request ID: REQ04
Service type: Security
Description: Sighting of a weapon
Requester name: Student A
Priority (Emergency / Standard / Low): Emergency
Service request added.


Priority Service Queue
----------------------
1. Add service request
2. Serve next request
3. Peek next request
4. Display all queues
0. Back

>> 1


Add Service Request
-------------------
Request ID: REQ05
Service type: Maintenance
Description: Hole in door
Requester name: Student B
Priority (Emergency / Standard / Low): Low
Service request added.
```

```
Priority Service Queue
----------------------
1. Add service request
2. Serve next request
3. Peek next request
4. Display all queues
0. Back

>> 4


All Priority Queues
-------------------

Emergency Queue:
  Request ID: REQ03 | Priority: Emergency | Service: Medical | Requester: Exam moniter | Description: Student passed out in exam |
  Request ID: REQ04 | Priority: Emergency | Service: Security | Requester: Student A | Description: Sighting of a weapon |

Standard Queue:
  Request ID: REQ01 | Priority: Standard | Service: IT Support | Requester: Professor Steve | Description: Laptop not working |

Low Queue:
  Request ID: REQ02 | Priority: Low | Service: Cleaning | Requester: Hallway Moniter | Description: Water spill in hallway |
  Request ID: REQ05 | Priority: Low | Service: Maintenance | Requester: Student B | Description: Hole in door |
```

5 different service requests were added at Emergency, Standard, and Low priority levels. The output display confirms that each service request was placed in the correct priority order, with Emergency first, then Standard, then Low. Furthermore, within each priority level, the priorities were dealt with according to the FIFO order of queues (the first request put into a priority level was the first dealt with)

5. **Fast lookup demo: Show insert and lookup operations on the building/resource store, including a lookup for a key that does not exist.**

```
Fast Building / Room Lookup
----------------------------
1. Insert building
2. Delete building
3. Lookup building
4. Insert room
5. Delete room
6. Lookup room
7. Show all buildings
0. Back

>> 3


Lookup Building
---------------
Building ID: arts
ID: ARTS
Name: ARTS
Location: (2, 5)
Rooms:
  ARTS-101 | capacity=40 | type=lecture
  ARTS-201 | capacity=25 | type=lab
```

```
Fast Building / Room Lookup
----------------------------
1. Insert building
2. Delete building
3. Lookup building
4. Insert room
5. Delete room
6. Lookup room
7. Show all buildings
0. Back

>> 6


Lookup Room
-----------
Room ID (press Enter to cancel): hunter common
No exact room found for "hunter common".
Rooms found containing that text:
  HUNTER_COMMONS-101 | capacity=40 | type=lecture
  HUNTER_COMMONS-201 | capacity=25 | type=lab
Please enter the full correct room ID.

Room ID (press Enter to cancel): HUNTER_COMMONS-101
Room ID: HUNTER_COMMONS-101
Capacity: 40
Type: lecture
```

The screenshots above demonstrate the fast building and room lookup features using a hash table. The screenshot on the left shows building lookup, where the system successfully retrieves building information even when the user enters the ID in lowercase by normalizing the input.

The screenshot on the right shows room lookup, where the system handles partial and case-insensitive input, suggests matching rooms when an exact ID is not found, and allows the user to select the correct room.

```
Fast Building / Room Lookup
---------------------------
1. Insert building
2. Delete building
3. Lookup building
4. Insert room
5. Delete room
6. Lookup room
7. Show all buildings
0. Back

>> 3


Lookup Building
---------------
Building ID: BUSINESS
Invalid building ID.
Valid building IDs:
ADMIN, ARTS, BOOKSTORE, ENG_BLOCK, GYM, HUNTER_COMMONS, ICT, KINESIOLOGY, LIBRARY, MACHALL, PARKADE, RESIDENCE, SCI_A, SCI_B, STUDENT_UNION
Building ID:
```

This screenshot demonstrates input validation in the building lookup feature. When an invalid building ID is entered, the system prompts the user again and displays a list of valid building IDs, ensuring correct and efficient lookup.

```
Fast Building / Room Lookup
---------------------------
1. Insert building
2. Delete building
3. Lookup building
4. Insert room
5. Delete room
6. Lookup room
7. Show all buildings
0. Back

>> 6


Lookup Room
-----------
Room ID (press Enter to cancel): BUSINESS-101
No exact room found for "BUSINESS-101".
No rooms found containing "BUSINESS-101".
Please try again.

Room ID (press Enter to cancel):
Room lookup cancelled.
```

This screenshot demonstrates input validation in the room lookup feature. When a non-existent room ID is entered, the system informs the user that no matching rooms were found and allows them to retry or cancel the lookup operation.

```
>> 1


Insert Building
---------------
Building ID: BUSINESS
Building name: Business Building
Location x-coordinate: 6
Location y-coordinate: 7
Building inserted.


Fast Building / Room Lookup
----------------------------
1. Insert building
2. Delete building
3. Lookup building
4. Insert room
5. Delete room
6. Lookup room
7. Show all buildings
0. Back

>> 3


Lookup Building
---------------
Building ID: BUSINESS
ID: BUSINESS
Name: Business Building
Location: (6, 7)
Rooms:
  none
```

```
>> 4


Insert Room
-----------
Building ID: BUSINESS
Room ID: BUSINESS-101
Capacity: 41
Room type: Lecture Hall
Room inserted.


Fast Building / Room Lookup
----------------------------
1. Insert building
2. Delete building
3. Lookup building
4. Insert room
5. Delete room
6. Lookup room
7. Show all buildings
0. Back

>> 6


Lookup Room
-----------
Room ID (press Enter to cancel): BUSINESS-101
Room ID: BUSINESS-101
Capacity: 41
Type: Lecture Hall
```

These screenshots demonstrate the insert and lookup functionality for buildings and rooms. The screenshot on the left shows inserting a new building and verifying it through lookup, while the screenshot on the right shows inserting a room into that building and successfully retrieving it using the room lookup feature.

```
PS C:\Users\Owner\OneDrive\Desktop\338FinalProject> & C:/Python314/pytho
finalproject/src/building_lookup_test.py

Fast Building Lookup Performance Demo
========================================
    10 records -> 0.00004931 ms per lookup
   100 records -> 0.00004490 ms per lookup
  1000 records -> 0.00005215 ms per lookup
 10000 records -> 0.00005274 ms per lookup
 50000 records -> 0.00004675 ms per lookup

Conclusion: Lookup time remains nearly constant as dataset size grows.
```

Additionally, to demonstrate that lookup performance is independent (or nearly independent) of the number of stored records, we inserted 10, 100, 1,000, 10,000, and 50,000 building records, then repeatedly performed lookups on a target building and calculated the average time per lookup. As shown in the screenshot, the lookup times remain very similar across all dataset sizes. This demonstrates that the lookup operation is nearly independent of the number of stored records, which is consistent with the average-case O(1) performance of a hash-table-based dictionary.

6. **Request pipeline demo: Show at least 10 requests being enqueued and dequeued in arrival order.**

```
Enqueued: Request #1
Enqueued: Request #2
Enqueued: Request #3
Enqueued: Request #4
Enqueued: Request #5
Enqueued: Request #6
Enqueued: Request #7
Enqueued: Request #8
Enqueued: Request #9
Enqueued: Request #10
Enqueued: Request #11
Enqueued: Request #12
Enqueued: Request #13
Enqueued: Request #14
Enqueued: Request #15
Enqueued: Request #16
Enqueued: Request #17
Enqueued: Request #18
Enqueued: Request #19
Enqueued: Request #20
```

```
-Processing in arrival order-

Processing (1): Request #1
Processing (2): Request #2
Processing (3): Request #3
Processing (4): Request #4
Processing (5): Request #5
Processing (6): Request #6
Processing (7): Request #7
Processing (8): Request #8
Processing (9): Request #9
Processing (10): Request #10
Processing (11): Request #11
Processing (12): Request #12
Processing (13): Request #13
Processing (14): Request #14
Processing (15): Request #15
Processing (16): Request #16
Processing (17): Request #17
Processing (18): Request #18
Processing (19): Request #19
Processing (20): Request #20
```

This screenshot demonstrates the enqueue and dequeue functionality for processing requests. The top of the screenshot shows requests being enqueued, and the bottom part of the screenshot shows the requests being processed (i.e., dequeued) in arrival order.